

8.5 B+ Tree: structure and Capacity

ISAM Falls Short For (Dynamic) Databases

- Limitations: When database changes tree can become quite "uneven".
- With long and short paths at different parts of the tree.
 - search time is fast or slow - unpredictable.
- Nodes can become full or empty.
 - Full: Insertion requires overflow
 - Empty: Waste disk storage space.
- What's missing?
 - Ability to maintain effectiveness under dynamic changes:
 - Balance of tree height.
 - Utilization of nodes (disk blocks)

B-Tree: Indexing Dynamic Data by Self Balancing

- A self-balancing tree data structure for indexing dynamic data.
- One of the most popular data structure in computer science.
- Support update effectively: Maintain balanced structure
 - All root-to-leaf paths are of the same length.
- Allow uniform accesses in logarithmic time.
 - $O(\log(n))$ for accessing a dataset of n items.
- Optimized for large block-oriented access of data.
 - Good for accessing data in external memory.

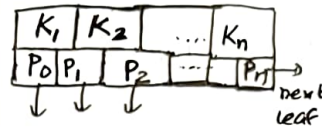
B+ tree is a small variation / enhancement of the original B-tree.

We discuss B+ tree

B+ Tree Node Structure: Like ISAM, But...
Not too empty (and Not too full)

- Parameter d , the degree (or order)
 - $d = 1.5, 2, 2.5, \dots$
- Each node has n keys and $n+1$ pointers.

• Leaf Node:



Internal Node:

almost same



- Max. number of keys: $n = 2d$
- Min. number of keys: $n \approx d$, i.e., 50% of the max.
 - (case 1) Leaf node: $n = \lceil d \rceil$ (thus, can be slightly larger than 50%)
 - (case 2) Internal node: $n = \lfloor d \rfloor$ (thus, can be slightly smaller than 50%)
- (Except) root: 1

B+ Tree Node capacity and Fanout

- How large is d ?
- Example:
 - Key size = 4 bytes
 - Pointer size = 8 bytes
 - Block size = 4096 bytes
- $2d \times 4 + (2d+1) \times 8 \leq 4096$ (depends on block size)
- $d = 170$
- $n = 2d = 340$
- Fanout (max) $F = n+1 = 341$ (no. of pointers)

How large data can B+ Tree Handle?

- Typical degree: 100. Typical fill-factor: 67%.
- Average fanout = 133.
- Can index large data with a few levels.

Height h	Number of Tuples N
1	433
2	17,689
3	2,352,637
4	312,900,721
5	41,615,795,893
6	41,615,795,893 5,534,900,853,769

- Can hold top levels in memory

Height h	Total Blocks	size(KB) for 4KB blocks
1	1	4
2	134	536
3	17,823	71,292
4	2,370,460	9,481,840

B+ Tree Pointers: How to label pointers (for what they lead to)?

- Pointer labelling: same as ISAM
- (case 1) Leaf node: P_i points to tuples with Key K_{i+1}
 - With last pointer P_n pointing to next leaf block.
- (case 2) Internal node: P_i points to index blocks of keys $[K_i, K_{i+1})$

(59)

8.6 B+ Tree: Operations

- Lookup tuples by a search key
 - Point lookup: grade = 80
 - Range lookup: grade > 80
- Making changes:
 - Insertion of a tuple
 - Deletion of a tuple
 - Update of a tuple = deletion then insertion

B+ Tree: Lookup

- Point queries: ... WHERE grade = 80
 Lookup(key) $\rightarrow$ pointer to tuple
 Same as in ISAM:
 - start at root
 - follow pointers by the ranges they represent.
 - until reach a leaf node.
 - check if key is there.
- Range queries: ... WHERE grade > 80
 Lookup(>key) $\rightarrow$ pointers to tuples
 - locate key or the closest value greater than key as in Lookup(key)
 - Then sequentially traverse following "next-leaf" pointers.

Maintaining a B+ Tree under Updates:

The Principle of Local changes

Principle: When a node becomes too full or too empty, perform "local" changes to maintain the tree in a good shape.

- Choose a suitable local change
 - A "local" change should involve only siblings: split, re-distributing pointers, merge.
 - Multiple actions are possible - solutions are not unique.
- Make the local change, so that:
 - Every node satisfies the node capacity requirement.
 - Every node has some $n+1$ pointers labeled by n keys.
- A local change may trigger more local changes recursively at parents.
 - so the changes will propagate up the tree.
- After the local changes, adjust keys, so that:
 - Every key $k = \text{MIN}(\text{keys } k' \text{ in the right-pointer subtree})$
 - Every key $k > \text{keys } k' \text{ in the left-pointer subtree.}$

B+ Tree: Insertion

(60)

Insert (K, P) for a new tuple with key K at pointer P .

- Lookup the leaf node N where K belongs.
- Insert K, P to N .
- If node becomes too full:
 - split node N into two: N and N' (with pointer P').
 - Insert P' to parent (recursively).
 - (variation) You can also just re-distribute pointers to neighbours.
- Adjust keys so that every key $k = \text{MIN}(\text{key } k' \text{ in the right-pointer subtree})$.
- If root splits, the tree adds one level.
 - The new root has 1 key only.
 - That's why root is allowed to have fewer keys than d .

B+ Tree: Deletion

Delete (K) for an existing tuple with key K .

- Lookup the leaf node N where K belongs.
- Delete K, P from N :
 - Remove P and the corresponding key K from N .
- If node becomes too empty:
 - Redistribute (move) pointers from a sibling ~~node N' (with pointer P')~~.
 - Merge node N with a sibling node N' (with pointer P')
 - Delete P' from parent (recursively).
- Adjust keys so that every key $k = \text{MIN}(\text{keys in } k' \text{'s right-pointer subtree})$
- If root had only one key and is now deleted, the tree reduces one level.